

Predicting Program Correctness By Ensemble Semantic Entropy

Yunxiang Wei
yunxwei@zju.edu.cn
Zhejiang University
China

Tianlin Li
tianlin001@buaa.edu.cn
Beihang University
China

Yuwei Zheng
By2506437@buaa.edu.cn
Beihang University
China

Yanni Dong
yannidong@outlook.com
University of Twente
Netherlands

Aishan Liu
liuaishan@buaa.edu.cn
Beihang University
China

Qiang Hu
qianghu@tju.edu.cn
Tianjin University
China

Xiaoyu Zhang
xiaoyu.zhang@ntu.edu.sg
Nanyang Technological University
Singapore

Mingfei Cheng
snowbirds.mf@gmail.com
Singapore Management University
Singapore

Jian Yang
jiayang@buaa.edu.cn
Beihang University
China

Abstract

Large language models (LLMs) have demonstrated remarkable capabilities in generating programs from natural language descriptions, yet ensuring their correctness without an external oracle remains a critical challenge. To solve the challenge, existing methods often rely on uncertainty estimation, measuring the consistency of semantics or execution behaviors across multiple samples generated by a single model. However, we observe that a single model can often converge to a consistent but incorrect solution, rendering such consistency-based proxies ineffective. To address this, we propose **Ensemble Semantic Entropy (ESE)**, which estimates uncertainty by evaluating the consistency of samples aggregated across an ensemble of models. Experiments on LiveCodeBench demonstrate that ESE correlates more strongly with program correctness than single-model semantic entropy. Notably, in selective generation tasks with strict false-positive rate constraints, ESE improves prediction accuracy by **53.4%**. Furthermore, by leveraging ESE as the decision signal, we propose a cascading test-time scaling framework CAS, which maintains performance while reducing FLOPs by **64.9%** compared to single-model scaling, offering a new perspective on balancing parameter and inference scaling.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

large language models, code generation, uncertainty estimation, semantic entropy, test-time scaling

1 Introduction

Large Language Models (LLMs) have shown strong potential in code generation tasks. For instance, Coding assistants such as GitHub Copilot and Cursor can automatically complete code snippets or generate programs from natural language instructions [8, 31], while agentic tools like Claude Code can further perform complex software engineering tasks, such as repository-level development [2]. As these autonomous tools are increasingly adopted in practical

development practice, LLM-generated code is correspondingly becoming deeply integrated into modern software systems.

However, despite their increasing deployment, LLM-generated programs cannot yet be fully trusted. Prior studies show that generated code may still contain hallucination, logical errors, and other defects that lead to incorrect behavior or security-relevant failures in realistic software development settings [24, 45]. While syntax errors and some superficial defects can often be detected by compiler tools, determining whether a syntactically valid program is functionally correct remains difficult. This motivates the need for methods that can predict the correctness of an LLM-generated program before deciding whether to rely on it. A natural approach is to validate candidate programs with existing or self-generated test cases, as seen in recent test-based generation and self-debugging methods [4, 7, 23]. Yet this strategy is often impractical in our setting: comprehensive external tests are rarely available in advance, especially for newly specified problems. Moreover, using LLM-generated tests as an oracle is inherently unreliable, as there is no guarantee that the generated tests themselves correctly capture the program specification, particularly for more challenging tasks [7, 10, 21, 35].

Recently, utilizing *uncertainty estimation* methods to predict program correctness has gained prominence. Unlike approaches that rely on external oracles, these methods predict correctness from the model’s inherent certainty of the generated program. Concretely, these methods typically sample multiple programs from a model and measure the consistency in the semantic space as the uncertainty of the model to the generated programs [17, 28]. For instance, [32] cluster the sampled programs by their I/O behavior on an LLM-generated test suite and use the size of the largest cluster as a certainty score. Valentin et al. [39] formalize the probability that two independently sampled programs produce different I/O behavior as incoherence score and use it as a lower bound on incorrectness. Sharma and David [36] further use symbolic execution to replace empirical execution behavior on a limited test suite and calculate uncertainty over the induced semantic space. Overall, these approaches are all grounded in the assumption that low uncertainty indicates the output program is likely correct, while high uncertainty is indicative of incorrectness.

However, we observe that although previous uncertainty measures correlated well with incorrectness, as it reflects divergent outputs across multiple samples for a given output, the correlation with correctness is not guaranteed when uncertainty is low (i.e., the model generates programs with consistent semantic behavior). Specifically, a model may exhibit *overconfidence*, causing the generated programs to converge to a consistent but *incorrect* behavior [13, 15, 37]. We illustrate this with a motivating example in Section 3 where a model consistently reproduces identical erroneous logic across syntactically diverse generations. This phenomenon causes existing single-model uncertainty metrics to yield *False Positives* (i.e., estimating low uncertainty for incorrect code), failing to effectively distinguish between “confident correctness” and “confident error”.

To address these limitations, relying on the uncertainty estimation of a single model is inherently insufficient. Our core insight is that while an individual model may confidently converge to a specific error mode, different models typically exhibit distinct failure patterns. Consequently, when multiple models reach a semantic consensus, the credibility of the solution is significantly higher than that derived from single-model self-consistency. Driven by this insight, we propose **Ensemble Semantic Entropy (ESE)**, a novel uncertainty estimation method that aggregates samples from multiple diverse models and performs clustering and entropy calculation in the semantic space. By clustering ensembled responses, ESE captures not only the uncertainty within individual models but also the uncertainty arising from distributional differences between models, thereby achieving oracle-free mutual calibration [18, 26].

Experiments on LiveCodeBench [14] demonstrate that ESE exhibits stronger correlation with program functional correctness than single-model semantic entropy. Notably, in selective generation tasks where model chooses to accept or reject an answer based on uncertainty signal, ESE yields **53.4%** improvement in prediction accuracy under strict False Positive Rate (FPR) constraints. Furthermore, we leverage ESE as a reliable decision signal to design CAS, a Cascading Test-Time Scaling framework [19, 20, 34, 43]. This framework dynamically evaluates generation quality, allowing smaller models to resolve problems where uncertainty is low and escalating to larger models only when uncertainty is high. Results show that compared to standard Best-of- N scaling, our framework preserves code generation performance while reducing FLOPs consumption by **64.9%**, offering a new perspective on balancing Parameter Scaling and Inference Scaling.

Our primary contributions are summarized as follows:

- We propose Ensemble Semantic Entropy (ESE), which effectively quantifies uncertainty in code generation by aggregating outputs from diverse models within the program semantic space.
- We validate the ability of ESE to predict program correctness in various experiments, demonstrating its stronger correlation with correctness and better performance in selective generation tasks.
- We introduce an ESE-based Cascading Test-Time Scaling Framework CAS, enabling dynamic resource allocation that significantly reduces inference costs while maintaining generation performance.

2 Preliminaries

2.1 Problem Setup

We define the code generation task as generating a program y given a prompt x . A Code LLM parameterized by θ induces a conditional distribution $p(y | x, \theta)$ over the space of programs $\mathcal{Y}$.

For the generated program, incorrectness can arise from two sources. The first is *syntax error*, where the generated code cannot be parsed or compiled. Such errors can typically be identified with compilers and are not the focus of this work. The second is *functional incorrectness*, where a syntactically valid program executes but does not satisfy the intended specification. Since our goal is to estimate uncertainty for correctness prediction beyond simple syntax checking, in the remainder of this paper we restrict $\mathcal{Y}$ to the space of syntactically valid programs.

For a given specification x , there exists a target correct function f_x^* . A generated program $y \in \mathcal{Y}$ is considered correct if and only if it functionally matches f_x^* , regardless of its implementation details. In this work, we focus on code generation tasks with a unique target behavior for each specification, excluding open-ended problems that admit multiple functionally distinct correct solutions.

Our primary goal is to employ *uncertainty estimation* to predict program correctness. Concretely, for a syntactically valid generated program y , we aim to derive an uncertainty score $\mathcal{U}(x, y)$ that correlates with the probability that y is functionally incorrect. A high uncertainty score should therefore indicate a high risk that the generated program fails to match the target function f_x^* .

2.2 Bayesian Uncertainty Quantification

From a Bayesian perspective, we treat the model parameters θ as a random variable. Given a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, training yields a posterior distribution $p(\theta | \mathcal{D})$. For an input x , the predictive distribution is obtained by marginalizing over the posterior:

$$p(y | x, \mathcal{D}) = \int p(y | x, \theta) p(\theta | \mathcal{D}) d\theta. \quad (1)$$

The total predictive uncertainty is classically quantified using the Shannon entropy H of this predictive distribution. Let Y denote the random variable for the output sequence:

$$H[Y | x, \mathcal{D}] = - \sum_{y \in \mathcal{Y}} p(y | x, \mathcal{D}) \log p(y | x, \mathcal{D}). \quad (2)$$

2.3 Semantic Entropy

Based on the information-theoretic decomposition of uncertainty [16], the total entropy can be expressed as:

$$H[Y | x, \mathcal{D}] = \underbrace{\mathbb{E}_{\theta \sim p(\theta | \mathcal{D})} [H[Y | x, \theta]]}_{\text{Aleatoric Uncertainty}} + \underbrace{I(Y; \Theta | x, \mathcal{D})}_{\text{Epistemic Uncertainty}}, \quad (3)$$

where Θ is the random variable representing model parameters.

Standard uncertainty methods typically estimate these quantities in the token space $\mathcal{Y}$. While in code generation tasks, many distinct sequences can implement the same algorithm. Consequently, the standard aleatoric term $H[Y | x, \theta]$ tends to overestimate uncertainty by capturing trivial lexical variations rather than meaningful

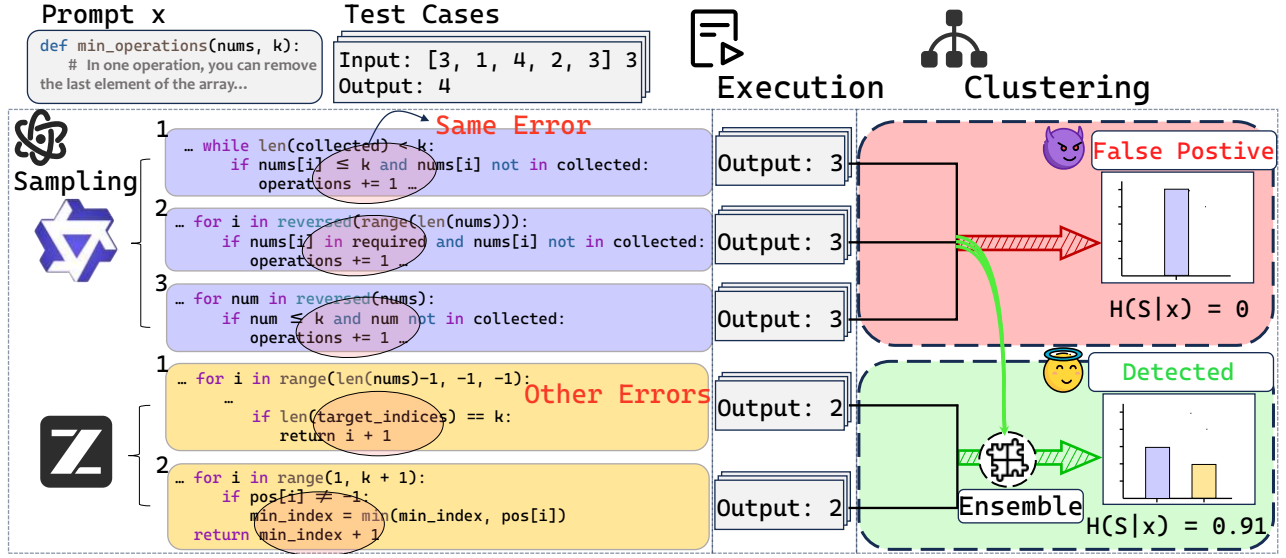


Figure 1: Calculation of Ensemble Semantic Entropy on a motivating example. The problem requires traversing an array `nums` backwards to collect integers from 1 to `k` with minimal steps. Qwen3-8B generates three programs that share the same incorrect semantics (only counting numbers within $[1, k]$), resulting in zero semantic entropy. GLM4-9B, however, generates programs that incorrectly output indices in forward order. By aggregating these five programs, the ensemble reveals the semantic disagreement, resulting in high uncertainty that correctly flags the error, effectively avoiding the false positive.

semantic ambiguity. Therefore uncertainty over *functional semantics* is often more relevant than uncertainty over specific token sequences.

Semantic Entropy (SE) [17] shifts the focus from raw tokens to semantic concepts via clustering samples first. Let $\phi : \mathcal{Y} \rightarrow \mathcal{S}$ be a semantic mapping function such that sequences y sharing the same meaning are grouped into a semantic cluster $s \in \mathcal{S}$ (i.e., $\phi(y) = s$). In practice, ϕ can be defined by execution based equivalence, which mapped programs into the same cluster if they produced identical outputs on a test set $\mathcal{T}$. Let a random variable S over the semantic space $\mathcal{S}$. The probability of a semantic cluster s under a fixed model θ is given by aggregating the probabilities of all samples belonging to that cluster:

$$p(s | x, \theta) = \sum_{y: \phi(y)=s} p(y | x, \theta). \quad (4)$$

The Semantic Entropy for a single model θ is defined as the entropy of the semantic variable S :

$$H_{SE}(x; \theta) := - \sum_{s \in \mathcal{S}} p(s | x, \theta) \log p(s | x, \theta). \quad (5)$$

3 Motivating Example

The premise of using standard SE for correctness prediction is that correctness usually converges to a single semantic behavior, while errors are diverse. However, this assumption can fail when a model exhibits **overconfidence**, namely, when it repeatedly produces semantically consistent but functionally incorrect programs.

Figure 1 illustrates a concrete example of this phenomenon. The problem requires traversing an array `nums` backwards to build a set. The Qwen3-8B model generates three programs that, while

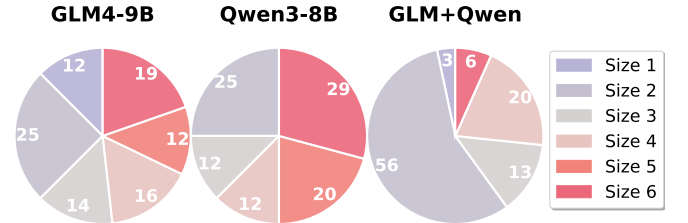


Figure 2: Comparison of the distribution of the largest cluster sizes in incorrect problems. Single models (GLM4-9B, Qwen3-8B) frequently show high consistency in incorrect answers, whereas the ensemble substantially reduces this spurious consistency.

implementing the backward traversal differently, share the exact same semantics with same error (missing the final element in total steps count). Since the three programs are functionally equivalent, they are mapped to the same semantic cluster. Thus the single-model SE is 0, falsely indicating that the model is confident about the correctness of the generated programs.

Overconfidence is prevalent in incorrect generations. To quantify the prevalence of this issue, we analyzed the behavior on the LiveCodeBench easy split as shown in Figure 2. We tested the GLM4-9B and Qwen3-8B models by generating 6 programs per problem and using the problem’s private tests as a comprehensive test $\mathcal{T}$ for execution based clustering. Programs are grouped into the same cluster if they procured identical outputs on all tests in $\mathcal{T}$. Focusing on problems where at least three programs were incorrect,

we measured the size of the largest cluster. The results show that for GLM4-9B and Qwen3-8B, 31% and 49% of these problems, respectively, exhibited a dominant incorrect cluster with a size greater than or equal to 5. In these instances where models are confidently wrong, single-model SE fails to distinguish them from correct cases because the estimated uncertainty remains extremely low.

Different models do not share error patterns. However, we find that such concentrated error patterns are not typically shared across different models. We constructed an ensemble by sampling 3 programs from GLM4-9B and 3 from Qwen3-8B, forming a set of 6 programs for behavioral clustering. Under the same setting, the proportion of problems with a dominant cluster larger than 5 samples was dramatically reduced to 6%.

This phenomenon can be understood by applying the uncertainty decomposition in Eq. (3) to the semantic variable $S = \phi(Y)$:

$$H[S \mid x, \mathcal{D}] = \mathbb{E}_{\theta \sim p(\theta \mid \mathcal{D})} [H[S \mid x, \theta]] + I(S; \Theta \mid x, \mathcal{D}). \quad (6)$$

The standard SE in Eq. (5) evaluates $H[S \mid x, \theta]$ at a single fixed model and therefore captures only within-model semantic dispersion. When a model is internally consistent yet concentrates on an incorrect semantic cluster, this quantity can remain low even though the generated program is wrong. In this sense, single-model SE reflects self-consistency rather than correctness [30]. To estimate the full uncertainty, the missing component is the cross-model disagreement term $I(S; \Theta \mid x, \mathcal{D})$, namely semantic epistemic uncertainty. Since different models often do not share the same error pattern, aggregating their semantic distributions may recover part of the missing epistemic uncertainty through disagreement between models on the incorrect programs. This observation motivates estimating semantic uncertainty from an ensemble of diverse models, which we formalize in Section 4.

4 Method

In this section, we present our method for predicting program correctness from **Ensemble Semantic Entropy (ESE)**. Motivated by the analysis in Section 3, our key idea is to estimate uncertainty from multiple models rather than from a single model alone, so as to capture not only within-model semantic dispersion but also cross-model disagreement.

We formulate entropy-based uncertainty estimation for code generation in three stages: (i) **semantic clustering**, which instantiates the semantic mapping for code generation; (ii) **entropy calculation**, which computes an uncertainty score from model-specific semantic distributions; and (iii) **correctness prediction**, which uses the uncertainty score to decide whether the generated programs are likely to be correct.

Correspondingly, we first specify how programs are mapped to semantic classes through execution-based functional equivalence, then compute uncertainty using ESE or its black-box variant EDSE, and finally apply a threshold to predict program correctness.

4.1 Semantic Clustering

In Section 2.3, Semantic Entropy is defined on a semantic space $\mathcal{S}$ through a mapping $\phi : \mathcal{Y} \rightarrow \mathcal{S}$. Here, we instantiate ϕ for code generation via functional equivalence.

Let $\mathcal{M} = \{\theta^{(\ell)}\}_{\ell=1}^L$ denote an ensemble of L code generation models. For code generation, semantic equivalence is defined by program behavior. Formally, viewing a program y as a function $f_y : \mathcal{I} \rightarrow \mathcal{O}$, two programs y and y' are functionally equivalent if $f_y(i) = f_{y'}(i), \forall i \in \mathcal{I}$. Accordingly, ϕ maps all functionally equivalent programs to the same semantic class.

As exact verification of functional equivalence is generally undecidable, we approximate this relation using a problem-specific set of test inputs $\mathcal{T}_x = \{t_1, \dots, t_{R_x}\}$. By executing a program y on $\mathcal{T}_x$, we obtain an output vector $\mathbf{o}_x(y) = [f_y(t_1), \dots, f_y(t_{R_x})]$. We then instantiate the semantic mapping by this execution behavior, so that $\phi(y) = \mathbf{o}_x(y)$. Under this approximation, two programs are assigned to the same semantic class if and only if they produce identical execution outputs on $\mathcal{T}_x$. Here, the semantic space $\mathcal{S}$ is given by the space of possible execution output vectors on $\mathcal{T}_x$.

4.2 Ensemble Semantic Entropy (ESE)

Given the semantic mapping ϕ , each model $\theta^{(\ell)}$ defines a semantic distribution over semantic classes:

$$p(s \mid x, \theta^{(\ell)}) = \sum_{y: \phi(y)=s} p(y \mid x, \theta^{(\ell)}). \quad (7)$$

Single-model Semantic Entropy (SE) computes uncertainty from one such distribution. In contrast, we first aggregate the semantic distributions from all models in the ensemble and then measures the entropy of the aggregated distribution.

We define the **Ensemble Semantic Probability** for a semantic class s as

$$p(s \mid x, \mathcal{M}) = \frac{1}{L} \sum_{\ell=1}^L p(s \mid x, \theta^{(\ell)}). \quad (8)$$

where we use uniform weights over the ensemble members. The **Ensemble Semantic Entropy (ESE)** is then defined as the entropy of this aggregated semantic distribution:

$$H_{\text{ESE}}(x) := - \sum_{s \in \mathcal{S}} p(s \mid x, \mathcal{M}) \log p(s \mid x, \mathcal{M}). \quad (9)$$

This definition also admits the following decomposition:

$$H_{\text{ESE}}(x) = \frac{1}{L} \sum_{\ell=1}^L H_{\text{SE}}(x; \theta^{(\ell)}) + \text{JSD}\left(p(\cdot \mid x, \theta^{(1)}), \dots, p(\cdot \mid x, \theta^{(L)})\right), \quad (10)$$

where JSD denotes the Jensen–Shannon divergence under uniform mixture weights, taken over the semantic distributions on $\mathcal{S}$. Hence, ESE consists of two parts: the average within-model semantic entropy and a cross-model disagreement term. The first term captures semantic dispersion within each individual model, whereas the second term captures how much the models disagree in semantic space.

This decomposition can explain why ESE is better aligned with the motivating example in Section 3. If a single model is internally consistent yet concentrates on one incorrect semantic cluster, its semantic entropy can remain low. However, if different models concentrate on different incorrect behaviors, the disagreement term increases, and the resulting ESE becomes high. In this sense, ESE extends single-model SE by incorporating a practical proxy for semantic epistemic uncertainty.

4.3 Ensemble Discrete Semantic Entropy (EDSE)

Computing H_{ESE} typically requires access to token-level likelihoods $p(y \mid x, \theta)$ (the *gray-box* setting), which is often infeasible for closed-source LLMs. To address this, existing work suggests a fully *black-box* variant, which is also valid in our framework [11].

For each model $\theta^{(\ell)}$, let $\{y_k^{(\ell)}\}_{k=1}^K$ denote its K sampled programs. After mapping each sample to its semantic class, we estimate the model-specific semantic distribution by empirical frequency:

$$\hat{p}(s \mid x, \theta^{(\ell)}) = \frac{1}{K} \sum_{k=1}^K \mathbb{I}[\phi(y_k^{(\ell)}) = s], \quad (11)$$

where $\mathbb{I}[\cdot]$ is the indicator function. Analogously to Eq. (8), we aggregate these empirical semantic distributions across the ensemble:

$$\tilde{p}_{\text{DSE}}(s \mid x) = \frac{1}{L} \sum_{\ell=1}^L \hat{p}(s \mid x, \theta^{(\ell)}). \quad (12)$$

The **Ensemble Discrete Semantic Entropy (EDSE)** is then defined as

$$H_{\text{EDSE}}(x) := - \sum_{s \in S} \tilde{p}_{\text{DSE}}(s \mid x) \log \tilde{p}_{\text{DSE}}(s \mid x). \quad (13)$$

Once the uncertainty score has been computed, we use this score to predict whether the generated programs are likely to be correct. Let $u(x)$ denote the uncertainty score for problem x , instantiated by either $H_{\text{ESE}}(x)$ or $H_{\text{EDSE}}(x)$. Since higher semantic uncertainty indicates greater dispersion or disagreement among sampled programs, we treat $u(x)$ as a proxy for the risk of functional incorrectness.

We adopt a threshold-based decision rule. Given a threshold τ , we predict that the sampled programs for x are reliable if $u(x) \leq \tau$, and unreliable otherwise. This defines a binary predictor that accepts low-uncertainty cases and rejects high-uncertainty cases. In later experiments, we evaluate how well this uncertainty-based decision rule supports correctness prediction.

5 Experiments

In this section, we evaluate ESE as an uncertainty signal for predicting program correctness in code generation task. To this end, we study the following research questions:

- **RQ1:** How strongly does ESE correlate with program correctness compared with single-model entropy-based uncertainty methods?
- **RQ2:** How do different semantic clustering methods affect the correlation between ESE and program correctness?
- **RQ3:** How accurately does ESE support selective generation with abstention under strict false-positive constraints?

5.1 Experimental Setup

We conduct experiments on **LiveCodeBench** [14] (Release v2), which consists of recent contest problems (e.g., AtCoder, Codeforces) equipped with rigorous, hidden test cases.

Following the formulation in Section 4, we compute uncertainty by first clustering sampled programs in semantic space and then applying an entropy calculation method to the induced distribution. Unless otherwise stated, we use the behavior-based clustering method introduced in Section 4.1, named *Func*, which groups

programs by their execution outputs on a problem-specific generated test set $\mathcal{T}_x^{\text{gen}}$; in our experiments, $\mathcal{T}_x^{\text{gen}}$ is pre-generated by Qwen3-Coder-30B. Under this default clustering, we compare five uncertainty scores: **Predictive Entropy (PE)**, **Semantic Entropy (SE)**, **Discrete Semantic Entropy (DSE)**, and our ensemble variants **ESE** and **EDSE**.

For each problem x we consider $M = 12$ sampled programs. In the single-model setting, all samples are drawn from one model. In the ensemble setting, the samples are evenly drawn from two models and merged before uncertainty computation.

We evaluate models from three families: GLM4-9B, the Qwen3 series (Qwen3-8B, Qwen3-14B, Qwen3-Coder-30B-A3B, Qwen3-Coder-485B-A35B), and the gpt-oss series (gpt-oss-20B, gpt-oss-120B) [29, 38, 41]. To investigate the effect of ensemble models on uncertainty calibration, we design two-model ensembles spanning both cross-family and within-family pairings. The cross-family settings probe whether model diversity provides a meaningful calibration signal, while the within-family settings test whether the gain persists when the paired models are likely to make more correlated errors. Specifically, for the smaller models Qwen3-8B and gpt-oss-20B, we pair each with a parameter-comparable model from a different family (respectively, GLM4-9B and Qwen3-Coder-30B). For mid-scale models (Qwen3-14B, Qwen3-Coder-30B, and gpt-oss-120B), we pair each with the smaller model from the same family to probe within-family ensembling (e.g., Qwen3-14B paired with Qwen3-8B). Finally, we also explore ensembling two strong large-scale models, Qwen3-Coder-485B with gpt-oss-120B. In all two-model settings, we draw $M/2$ samples from each model and merge them into a set of M programs, on which we compute uncertainty.

5.2 RQ1. Correlation with Program Correctness

To answer RQ1, we evaluate how these uncertainty scores correlate with program correctness. For each problem x , we evaluate all M sampled programs on the official LiveCodeBench tests and obtain per-sample correctness scores $\{c_m\}_{m=1}^M$, where each score is defined as the proportion of hidden tests passed. We then summarize correctness by the mean score $\bar{c}(x) := \frac{1}{M} \sum_{m=1}^M c_m$ and compute the Pearson correlation between $\bar{c}(x)$ and the corresponding uncertainty score $u(x)$ across problems. A stronger negative correlation indicates that higher uncertainty more faithfully signals lower functional correctness. The results are summarized in Table 1.

Ensemble semantic entropy provides the strongest correctness signal. Across the comparable settings, **ESE** and **EDSE** consistently yield the strongest or nearly strongest correlations with correctness. Relative to the corresponding single-model baselines, the gains are especially clear for cross-family ensembles: the magnitude improves from -0.6738 to -0.7400 for Qwen3-8B+GLM4-9B and from -0.5961 to -0.7122 for gpt-oss-20B+Qwen3-Coder-30B. Within-family ensembling yields smaller but still consistent improvements, such as from -0.6852 to -0.7037 for Qwen3-14B+Qwen3-8B and from -0.6686 to -0.6998 for Qwen3-Coder-30B+Qwen3-8B. This pattern suggests that cross-model disagreement provides information about correctness beyond single-model self-consistency.

Table 1: Pearson correlation coefficients and p-values between uncertainty and functional correctness on LiveCodeBench for each model (higher magnitude negative is better). We report Predictive Entropy (PE), Semantic Entropy (SE), and Discrete Semantic Entropy (DSE) under functional behavior clustering. Gray rows denote our proposed ensemble variants, Ensemble Semantic Entropy (ESE) and Ensemble Discrete Semantic Entropy (EDSE). Bold values indicate the best correlation coefficients, while underlined values indicate the second-best results.

Metric	GLM4-9B	Qwen3-8B	Qwen3-14B	Qwen3-Coder-30B	Qwen3-Coder-485B	gpt-oss-20B	gpt-oss-120B
PE	-0.0899 (4×10^{-2})	0.0619 (1×10^{-1})	0.2670 (3×10^{-53})	-0.1066 (1×10^{-2})	-0.4183 (5×10^{-26})	0.1986 (6×10^{-6})	-0.4320 (5×10^{-12})
SE	-0.7352 (2×10^{-101})	-0.6735 (3×10^{-81})	-0.6852 (1×10^{-75})	-0.6686 (2×10^{-67})	-0.6304 (5×10^{-58})	-0.5960 (2×10^{-50})	-0.6751 (1×10^{-67})
DSE	-0.7372 (2×10^{-102})	-0.6738 (6×10^{-81})	-0.6864 (8×10^{-76})	-0.6682 (2×10^{-67})	-0.6304 (3×10^{-58})	-0.5961 (2×10^{-50})	-0.6694 (3×10^{-67})
ESE	–	<u>-0.7391</u> (9×10^{-90})	<u>-0.7029</u> (1×10^{-77})	-0.6998 (1×10^{-70})	-0.6641 (5×10^{-63})	<u>-0.7106</u> (3×10^{-80})	-0.6736 (2×10^{-43})
EDSE	–	-0.7400 (2×10^{-89})	-0.7037 (2×10^{-77})	<u>-0.6992</u> (2×10^{-70})	<u>-0.6638</u> (2×10^{-64})	-0.7122 (9×10^{-80})	<u>-0.6750</u> (6×10^{-44})

Semantic entropy is much more correlated with program correctness than token-level predictive entropy. All semantic-entropy variants are strongly negatively correlated with correctness, typically around -0.6 to -0.74. In contrast, PE stays close to 0 and is even positive for some models, such as Qwen3-8B (0.0619), Qwen3-14B (0.2670), and gpt-oss-20B (0.1986). This gap is also reflected in statistical significance. For uncertainty methods based on semantic entropy, including SE, DSE, ESE, and EDSE, the p -values are consistently far below conventional thresholds, typically smaller than 10^{-40} . In contrast, the correlation of PE with program correctness is much less stable, with some models yielding only weak significance or even insufficient significance (e.g., $p > 10^{-2}$). Taken together, these results show that token-level predictive entropy is not a reliable correctness signal for code generation, whereas semantic entropy provides a substantially more reliable correlate of program correctness.

Ensembling matters more than refining single-model probability information. Notably, SE and DSE are almost indistinguishable across all backbones, while ESE and EDSE consistently improve over them. This suggests that the main gain comes from incorporating cross-model variation rather than from refining single-model probability information.

Answer to RQ1: ESE and EDSE generally show stronger correlation with program correctness than single-model uncertainty methods, indicating that ensemble uncertainty provides a stronger correctness signal.

5.3 RQ2. Effects of Different Clustering Methods

To answer RQ2, we study how the choice of clustering method affects the correlation between uncertainty and program correctness beyond the default *Func* setting. We consider five clustering methods in this study, including *NLI*, *Embed*, *BLEU*, *Symbolic*, and *Func*:

- *NLI*: uses DeBERTa-v2-xlarge-mnli to test bidirectional entailment between code pairs [17];
- *Embed*: clusters by distances in embedding space using SFR-Embedding-Code-400M_R;
- *BLEU*: uses CodeBLEU [33] to measure program similarity via a mixture of AST, control-flow, and n-gram overlap;

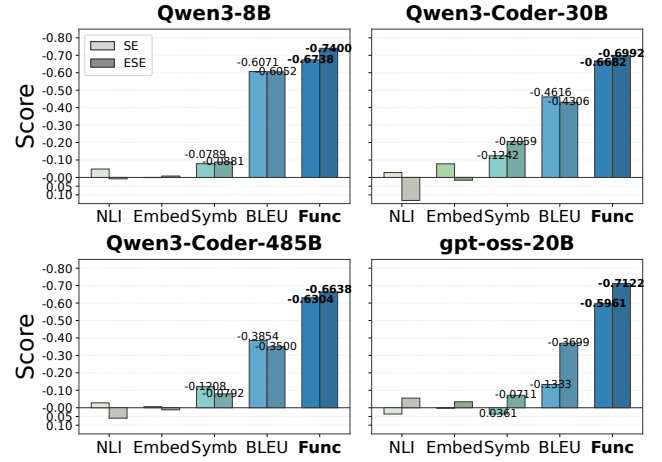


Figure 3: Pearson correlation coefficients between uncertainty and mean program correctness under different semantic clustering methods. For each method, the left bar denotes SE and the right bar denotes ESE. Results are reported for four representative backbones.

- *Symbolic*: applies CrossHair for symbolic execution [36], declaring two programs equivalent if no counterexample is found within the prescribed time budget. We set the per-path timeout to 5 seconds and the total timeout to 30 seconds;
- *Func*: the behavior-based method introduced in Section 4.1, which clusters programs by their execution traces on test inputs.

We report the Pearson correlation between uncertainty measures and mean correctness under each clustering choice of four representative models in Figure 3.

The choice of clustering method substantially affects uncertainty estimation. Figure 3 shows that the clustering method has a clear impact on the correlation between uncertainty and correctness. Among the five methods, *Func* consistently yields the strongest negative correlation across the representative backbones, while *NLI*, *Embed*, and *Symbolic* are generally much weaker. *BLEU* can partially capture algorithmic differences through code-structure overlap, yet it cannot reliably detect semantically critical

changes that preserve surface control flow while altering functional behavior; consequently, uncertainty computed under *BLEU* clustering remains weaker than the execution-grounded *Func* variant. Finally, while symbolic analysis can in principle support path-complete reasoning, its practical use in our setting is limited: it typically requires manually specified preconditions to constrain the input space, and without such preconditions it may explore invalid inputs (e.g., negative values used as array lengths). Moreover, for complex programs it frequently exceeds the time budget and is then forced to conservatively declare equivalence. This trend suggests that, for code generation, uncertainty estimation benefits most from a semantic partition grounded in executable functional behavior rather than superficial similarity.

Ensembling improves correlation under nearly all clustering methods. Although the absolute quality of the clustering method varies, the effect of ensembling remains broadly consistent. In Figure 3, ESE achieves a stronger correlation than the corresponding SE result for almost every clustering method and model. This result shows that the benefit of ensemble-based uncertainty is not tied to a single clustering choice: once a semantic partition is induced, aggregating multiple models generally improves the correlation between uncertainty and program correctness.

Answer to RQ2: Different clustering methods substantially affect the correlation between ESE and program correctness. Functional clustering yields the strongest correlation, while the correlation gain from ensembling remains consistent across nearly all clustering choices.

5.4 RQ3. Performance on Selective Generation Task

To answer RQ3, we evaluate whether the uncertainty signal can support accurate downstream decisions in selective generation tasks. In selective generation, the model should *abstain* when it is likely to produce functionally incorrect code. To predict the correctness of generated code, we compute an uncertainty score $u(x)$ from the semantic clusters of the M sampled programs on $\mathcal{T}_x^{gen}$. Given a threshold τ , we *accept* the prediction if $u(x) \leq \tau$ and *reject* it otherwise. Upon acceptance, we follow a majority-voting principle: we select the *largest* semantic cluster and output the *longest* program within that cluster as a deterministic selection rule. We then evaluate the selected program on the official LiveCodeBench tests and define it as pass if it succeeds (score = 1) and fail otherwise.

We evaluate performance by the trade-off between *False Positive Rate* (FPR), namely the fraction of fail cases that are incorrectly accepted, and *True Positive Rate* (TPR), namely the fraction of pass cases that are correctly accepted. We report TPR under strict FPR constraints of 5% and 10% (TPR@5FPR, TPR@10FPR), together with the overall accept/reject classification accuracy (Acc.) at the 10% FPR operating point. In addition to SE/DSE and their ensemble variants, we include **SE-NLI** as the original semantic entropy baseline that clusters samples using NLI-based entailment judgments. Results are shown in Table 2.

Ensemble-based uncertainty yields substantially better abstention decisions. As shown in Table 2, **ESE** substantially outperforms single-model uncertainty across all three evaluated backbones, and **EDSE** shows similarly strong performance. In particular, compared with **SE**, TPR@5FPR of **ESE** increases by 83%, 172%, and 49% for Qwen3-8B, Qwen3-Coder-30B, and gpt-oss-20B, respectively, while the corresponding classification accuracy improves by 28.5%, 53.2%, and 37.8%. **EDSE** attains the best results for Qwen3-8B, while remaining comparable to **ESE** on the other two backbones. These gains are consistent with the correlation results above: by incorporating cross-model disagreement, ensemble-based semantic entropy better detects confident but incorrect solutions that single-model self-consistency tends to miss, and therefore provides a more reliable signal for selective generation under strict false-positive constraints.

Answer to RQ3: ESE supports more accurate selective-generation decisions than single-model baselines under strict false-positive constraints, showing that its correlation advantage transfers to downstream decision making.

6 Application

We instantiate the uncertainty measure from Section 4 in a test-time scaling (TTS) pipeline for code generation, following the public/private test protocol used in programming contests and online judges. Given a problem specification x , let $\mathcal{T}_x^{pub}$ and $\mathcal{T}_x^{priv}$ denote the public and hidden test suites, respectively. A candidate program y can be executed on $\mathcal{T}_x^{pub}$ to detect obvious misunderstandings or runtime errors, while correctness is ultimately determined by passing $\mathcal{T}_x^{priv}$.

Existing TTS frameworks primarily scale inference-time compute, such as parallel sampling and iterative refinement, for a fixed model, whereas model capability can also be improved by scaling model parameters [1, 42]. In practice, scaling test-time compute alone often exhibits diminishing returns: substantially more computation may yield only marginal performance gains. Our goal is therefore to balance inference scaling and parameter scaling via a cascading mechanism: we start from cheaper models and escalate to stronger, more expensive models only when a decision module predicts that the current layer is unlikely to return a correct solution [44].

6.1 Cas: Multi-layer Cascading Framework

We propose an H -layer framework, Cas. Each layer $\ell \in \{1, \dots, H\}$ is associated with an ensemble of N_ℓ code generators $\mathcal{M}^{(\ell)} = \{\theta_j^{(\ell)}\}_{j=1}^{N_\ell}$, where $\theta_j^{(\ell)}$ denotes the model instance used to generate the j -th candidate. When all $\theta_j^{(\ell)}$ are identical, the layer reduces to standard single-model scaling. On input x , the pipeline starts at $\ell = 1$ and executes the following stages within each layer.

(i) **Parallel sampling.** We draw N_ℓ candidate programs in parallel, producing $\mathcal{Y}^{(\ell)} = \{y_j^{(\ell)}\}_{j=1}^{N_\ell}$, where $y_j^{(\ell)}$ is sampled from $\theta_j^{(\ell)}$.

(ii) **Iterative public-test debugging.** Each sampled program $y_j^{(\ell)}$ is allowed up to D_ℓ serial refinement steps, where $\theta_j^{(\ell)}$ uses feedback from executing $y_j^{(\ell)}$ on $\mathcal{T}_x^{pub}$ (e.g., failing cases or runtime

Table 2: Selective generation results on LiveCodeBench. We report TPR at fixed FPR thresholds (TPR@5FPR, TPR@10FPR) and the resulting classification accuracy (Acc.) at the 10% FPR constraint. We use SE-NLI, SE/DSE, and the proposed ESE/EDSE as uncertainty scores to decide whether to abstain. Bold values indicate the highest accuracy or TPR.

Method	Qwen3-8B			Qwen3-Coder-30B			gpt-oss-20B		
	Acc.	TPR@5FPR	TPR@10FPR	Acc.	TPR@5FPR	TPR@10FPR	Acc.	TPR@5FPR	TPR@10FPR
SE-NLI	0.4496	0.0500	0.1001	0.3145	0.0460	0.0920	0.1593	0.0576	0.1153
SE	0.6420	0.3961	0.6199	0.4085	0.1797	0.3600	0.4671	0.4050	0.8080
DSE	0.6398	0.3920	0.6139	0.4078	0.1787	0.3575	0.4689	0.4070	0.8106
ESE	0.8247	0.7246	0.7880	0.6258	0.4889	0.7737	0.6435	0.6041	0.8749
EDSE	0.8267	0.7283	0.7980	0.5903	0.4384	0.7569	0.6384	0.5983	0.8668

errors) to produce an edited program. We denote the resulting program by $\tilde{y}_j^{(\ell)}$. Let $\mathbb{I}[\tilde{y}_j^{(\ell)} \text{ passes } \mathcal{T}_x^{\text{pub}}]$ be the indicator of public-test success, and define the *public-pass ratio*

$$\lambda_\ell(x) := \frac{1}{N_\ell} \sum_{j=1}^{N_\ell} \mathbb{I}[\tilde{y}_j^{(\ell)} \text{ passes } \mathcal{T}_x^{\text{pub}}]. \quad (14)$$

We retain only the passing candidates $\tilde{\mathcal{Y}}_{\text{pass}}^{(\ell)} = \{\tilde{y}_j^{(\ell)} : \tilde{y}_j^{(\ell)} \text{ passes } \mathcal{T}_x^{\text{pub}}\}$.

(iii) Clustering-based decision and selection. Using the passing set $\tilde{\mathcal{Y}}_{\text{pass}}^{(\ell)}$, we compute semantic clusters induced by the generated test set $\mathcal{T}_x^{\text{gen}}$ and evaluate the ensemble semantic uncertainty $\text{ESE}_\ell(x)$. If $\tilde{\mathcal{Y}}_{\text{pass}}^{(\ell)} = \emptyset$ at a non-final layer, the current layer is rejected immediately and the pipeline proceeds to the next layer. Otherwise, a *cascade decision maker* compares a scalar score $S_\ell(x)$ against a threshold τ_ℓ to decide whether to accept the current layer or escalate the problem to layer $\ell + 1$. If accepted, a selection rule returns a single program from $\tilde{\mathcal{Y}}_{\text{pass}}^{(\ell)}$. The final layer $\ell = H$ omits the rejection option, ensuring termination.

6.2 Layerwise Cascade Decision Maker

The decision maker integrates two complementary signals: (1) **Plausibility**: represented by the public-pass ratio λ_ℓ (higher is better), indicating the models' overall ability to satisfy basic constraints. (2) **Consensus**: represented by semantic uncertainty (lower is better). High agreement among samples from diverse models suggests reliability.

We define a normalized uncertainty $\hat{u}_\ell(x)$ from $\text{ESE}_\ell(x)$. When the passing set induces at least two semantic clusters, we normalize by the maximum entropy under that partition: $\hat{u}_\ell(x) = \frac{\text{ESE}_\ell(x)}{\log |\hat{C}_\ell(x)|}$.

When $|\hat{C}_\ell(x)| \leq 1$, we set $\hat{u}_\ell(x) = 0$. The cascade score is then defined as

$$S_\ell(x) := \alpha_\ell \lambda_\ell(x) - (1 - \alpha_\ell) \hat{u}_\ell(x), \quad (15)$$

where $\alpha_\ell \in [0, 1]$ balances the two terms. The decision rule is $d_\ell(x) = \mathbb{I}[S_\ell(x) \geq \tau_\ell]$. A lower τ_ℓ encourages earlier acceptance at lower layers and lower cost, whereas a higher τ_ℓ imposes stricter requirements on both public-test success and cross-sample agreement, thereby escalating uncertain problems to higher layers.

6.3 Selection via Semantic Clustering

Upon acceptance at layer ℓ (or when $\ell = H$), the system must return a single program $\hat{y}$. We leverage the semantic partition $\hat{C}_\ell(x)$

computed during uncertainty estimation. As at most one cluster corresponds to the correct behavior on $\mathcal{T}_x^{\text{gen}}$, we assume that the correct behavior is more likely to be reproduced consistently across generators than a spurious behavior and simply apply *majority voting*:

$$c_\ell^*(x) := \arg \max_{c \in \hat{C}_\ell(x)} |c|. \quad (16)$$

The final output is then sampled uniformly from the dominant cluster $c_\ell^*(x)$.

Algorithm 1 summarizes the full CAS test-time scaling procedure.

Algorithm 1 CAS Framework

Require: Problem x , public tests $\mathcal{T}_x^{\text{pub}}$, generated tests $\mathcal{T}_x^{\text{gen}}$, layer ensembles $\{\mathcal{M}^{(\ell)}\}_{\ell=1}^H$, sampling budgets $\{N_\ell\}$, debug budgets $\{D_\ell\}$, cascade parameters $\{(\alpha_\ell, \tau_\ell)\}$.

Ensure: Final program $\hat{y}$

```

1: for  $\ell = 1 \dots H$  do
2:   Stage 1: Parallel Sampling
3:    $\mathcal{Y}^{(\ell)} \leftarrow \{y_j^{(\ell)} \sim \theta_j^{(\ell)}(\cdot|x)\}_{j=1}^{N_\ell}$  where  $\theta_j^{(\ell)} \in \mathcal{M}^{(\ell)}$ 
4:   Stage 2: Public-test Debugging
5:    $(\tilde{\mathcal{Y}}^{(\ell)}, \lambda_\ell) \leftarrow \text{DEBUGANDFILTER}(\mathcal{Y}^{(\ell)}, \mathcal{T}_x^{\text{pub}}, D_\ell)$ 
6:   Stage 3: Clustering & Uncertainty Estimation
7:    $(C_\ell, u_\ell) \leftarrow \text{UNCERTAINTYESTIMATE}(\tilde{\mathcal{Y}}^{(\ell)}, x)$ 
8:   Stage 4: Cascade Decision & Selection
9:    $S_\ell \leftarrow \alpha_\ell \lambda_\ell - (1 - \alpha_\ell) u_\ell$ 
10:  if  $S_\ell \geq \tau_\ell$  or  $\ell = H$  then
11:    return  $\text{SELECTMAJORITY}(C_\ell)$ 
12:  end if
13: end for
```

6.4 Evaluation

Experimental setting. We evaluate CAS ($H = 2$) on two target backbones, Qwen3-Coder-485B-A35B and gpt-oss-20B, and compare it with single-model test-time scaling strategies. The strongest baseline is $\mathbf{S}^*$ [20], which combines parallel sampling ($N = 20$), iterative refinement ($D = 3$), and best-of- N selection on a single model. In the cascading TTS framework CAS, the first layer is a lightweight ensemble of Qwen3-8B and Qwen3-Coder-30B-A3B, with 5 samples from each model. The second layer uses 15 samples from a stronger target backbone, either Qwen3-Coder-485B-A35B or gpt-oss-20B; we denote the resulting framework by $\text{Cas}(\text{Qwen})$

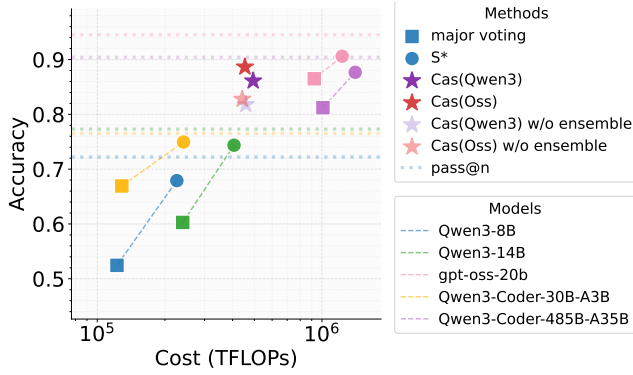


Figure 4: Accuracy-cost comparison on LiveCodeBench across four test-time scaling methods, Majority Voting, S^* , CAS (w/o ensemble), and CAS, over five model configurations. Pass@20 is shown as the corresponding selection upper bound for each model. Methods in the upper-left region exhibit better accuracy-cost trade-offs.

and Cas(Oss), respectively. We set $\alpha_1 = 0.5$ and $\tau_1 = 0.3$. For reference, we also report **Major Voting**, which applies semantic clustering and returns a program from the largest cluster, and CAS (w/o ensemble), which replaces the two-model ensemble in first layer with only Qwen3-Coder-30B-A3B. We report the accuracy on LiveCodeBench and overall inference cost of each method in Figure 4.

CAS preserves near-best performance while substantially reducing inference cost, yielding the best overall accuracy-cost trade-off. As shown in Figure 4, CAS lies closest to the upper-left frontier among the evaluated methods. On Qwen3-Coder-485B-A35B, Cas(Qwen) reaches 86.11% pass rate with cost 2.47, compared with 87.67% and 7.03 for S^* . On gpt-oss-20B, Cas(Oss) reaches 88.65% with cost 2.27, compared with 90.61% and 6.15 for S^* . Thus, CAS reduces cost by 64.9% and 63.1%, respectively, while keeping the accuracy gap to S^* within 1.56 and 1.96 percentage points. Compared with simple Majority Voting, CAS improves both pass rate and inference cost, indicating that augmenting semantic-clustering-based selection with iterative public-test debugging improves candidate quality substantially, while incurring only a modest additional cost. Moreover, replacing the first-layer ensemble with a single model causes a clear performance degradation, reducing pass rate by 4.31 and 5.87 percentage points for Cas(Qwen) and Cas(Oss), respectively. We next analyze these gains through the behavior of the cascade decision maker.

To further understand why CAS achieves high accuracy under low inference cost, we analyze the first-layer decision $d_1(x) = \mathbb{I}[S_1(x) \geq \tau_1]$, where $S_1(x) = \alpha_1 \lambda_1(x) - (1 - \alpha_1) \hat{u}_1(x)$ combines the public-pass ratio $\lambda_1(x)$ and the normalized uncertainty $\hat{u}_1(x)$. We first study the role of the threshold τ_1 , and then examine the contributions of the first-layer ensemble and the two decision signals. In these analyses, we report two metrics. **Exit@L1** denotes the fraction of problems for which $d_1(x) = 1$, i.e., the fraction of problems accepted at Layer 1 rather than escalated to Layer 2. **Pass@Exit** denotes the hidden-test pass rate of the selected programs among

Table 3: Effect of the first-layer threshold τ_1 on the trade-off between accuracy and inference cost (Qwen3-Coder-485B).

τ_1	Pass $\uparrow$	Cost $\downarrow$	Exit@L1 $\uparrow$	Pass@Exit $\uparrow$
0.20	79.21	1.90	80.98	81.84
0.30	86.11	2.47	69.80	92.13
0.40	86.50	2.92	61.57	93.31
0.50	86.69	3.25	55.29	94.68

these accepted problems. These two metrics characterize the trade-off made by the cascade decision maker: increasing Exit@L1 reduces inference cost by accepting more problems at the first layer, whereas increasing Pass@Exit indicates that the selected programs returned by those first-layer decisions are more reliable.

We first study how the threshold τ_1 controls the trade-off between acceptance rate and the quality of the selected program. We vary τ_1 on Cas(Qwen) while keeping the other settings unchanged. Table 3 reports the resulting overall pass rate, average inference cost, Exit@L1, and Pass@Exit.

$\tau_1 = 0.3$ provides the best balance between early-exit precision and inference cost. When τ_1 is reduced to 0.20, the cascade decision maker accepts too many problems at Layer 1 while these selected programs are ultimately incorrect. Exit@L1 rises to 80.98% and cost drops to 1.90, but Pass@Exit falls to 81.84% and the final pass rate drops sharply to 79.21%. When τ_1 increases beyond 0.30, the cascade decision maker becomes more conservative. Pass@Exit further improves from 92.13% to 93.31% and 94.68%, but many problems that could already be handled by the first layer are instead escalated to Layer 2, reducing Exit@L1 to 61.57% and 55.29% and increasing cost to 2.92 and 3.25, while the final pass rate improves only marginally to 86.50% and 86.69%. Therefore, it is reasonable to set $\tau_1 = 0.3$, as it provides the best balance between pass rate and inference cost.

We next examine the contribution of each component in the cascade decision maker. We consider three ablations: CAS (w/o ensemble), which replaces the first-layer ensemble with only Qwen3-Coder-30B-A3B; CAS (uncertainty only), which removes the public-pass term by setting $\alpha_1 = 0$, so that the decision score becomes

$$S_1^{\text{unc}}(x) = -\hat{u}_1(x), \quad (17)$$

with acceptance determined by a corresponding threshold on uncertainty; and CAS (public-pass only), which removes the uncertainty term by setting $\alpha_1 = 1$, so that

$$S_1^{\text{pub}}(x) = \lambda_1(x), \quad (18)$$

with acceptance determined solely by the public-pass ratio. Table 4 reports the resulting pass rate, cost, Exit@L1, and Pass@Exit on Cas(Qwen) and Cas(Oss).

Utilizing ensemble semantic entropy as decision signal improves early-exit reliability by reducing false-positive exits. Relative to full CAS, CAS (w/o ensemble) reduces the final pass rate from 86.11% to 81.80% on Qwen3-Coder-485B-A35B and from 88.65% to 82.78% on gpt-oss-20B, while lowering Pass@Exit from 92.13% to 86.98% and increasing Exit@L1 from 69.80% to 76.86%. In full CAS, the first-layer cascade decision maker evaluates ESE₁ computed from the ensemble of Qwen3-8B and Qwen3-Coder-30B-A3B,

Table 4: Ablation analysis of the CAS cascade decision maker on the two models. Pass rate is reported in %, cost is reported in 10^6 TFLOPs, Exit@L1 denotes the fraction of instances accepted at Layer 1, and Pass@Exit denotes the hidden-test pass rate among these Layer-1 exits.

Method	Qwen3-Coder-485B				gpt-oss-20B			
	Pass↑	Cost↓	Exit@L1↑	Pass@Exit↑	Pass↑	Cost↓	Exit@L1↑	Pass@Exit↑
CAS	86.11	2.47	69.80	92.13	88.65	2.27	69.80	92.13
CAS (w/o ensemble)	81.80	2.29	76.86	86.98	82.78	2.21	76.86	86.98
CAS (uncertainty only)	79.01	2.28	73.52	74.40	80.21	2.01	73.52	74.40
CAS (public-pass only)	68.82	2.73	65.09	68.37	69.93	2.13	65.09	68.37

whereas CAS (w/o ensemble) effectively relies on the semantic entropy of Qwen3-Coder-30B-A3B alone. As discussed in Section 5.4, single-model SE is more vulnerable to false-positive solutions: a model may generate candidate programs that are semantically consistent yet functionally incorrect, causing uncertainty to be underestimated and the problem to be accepted too early. This effect explains why CAS (w/o ensemble) yields higher Exit@L1 but lower Pass@Exit. By incorporating cross-model disagreement, ESE better identifies these false-positive solutions and escalates them to the stronger second layer, thereby improving the overall pass rate.

Both Plausibility and Consensus signals in the cascade decision maker are necessary, as they capture different failure modes. CAS (uncertainty only) and CAS (public-pass only) both underperform the full cascade decision maker for different reasons. CAS (uncertainty only), which relies only on uncertainty, reaches 79.01% and 80.21% final pass rate on the two backbones and lowers Pass@Exit to 74.40%. This result shows that low uncertainty alone is insufficient: for problems that the first layer cannot solve reliably, $\lambda_1(x)$ is often small, so the passing set $\tilde{\mathcal{Y}}_{\text{pass}}^{(1)}$ is also small. Any apparent consensus in such a limited set is therefore unreliable, and $\hat{u}_1(x)$ alone does not justify accepting the current layer.

CAS (public-pass only), which makes the Layer-1 acceptance decision using only the public-pass ratio, drops to 68.82% and 69.93% final pass rate, with Pass@Exit only 68.37%, the worst result among all decision-maker variants. This result shows that public tests alone are also insufficient: even when many candidates satisfy $\mathcal{T}_x^{\text{pub}}$, they may still occupy different semantic clusters on $\mathcal{T}_x^{\text{gen}}$ and disagree on hidden-test behavior, reflecting substantial uncertainty on these candidate programs. Therefore, the construction of $S_1(x)$ is justified because $\lambda_1(x)$ and $\hat{u}_1(x)$ capture different failure modes, and the current layer is reliable only when the public-pass ratio is sufficiently high and the semantic uncertainty is sufficiently low.

7 Related Work

Uncertainty Estimation in Code Generation. Uncertainty estimation serves as a proxy for detecting hallucinations and predicting the correctness of responses from Large Language Models (LLMs). Concurrently, mainstream estimation techniques have evolved from token-level statistics to sampling-based consistency methods, most notably Semantic Entropy (SE) [11, 17, 28, 30], which evaluates uncertainty by measuring entropy across clusters of semantically

equivalent outputs. In the domain of code generation, such equivalence is rigorously defined by functional behavior; recent works utilize generated test cases [32, 39] or symbolic execution [36] to verify output consistency. Notably, [9] explores triangulation for open-ended problems, while our work mainly focuses on problems with a unique correct solution.

Test-Time Scaling for Code Generation. Test-time scaling (TTS) posits that increasing the computational budget during inference can significantly enhance model performance. In the domain of code generation, TTS strategies generally follow three main paradigms: (i) **Extending Reasoning:** Extending reasoning length with techniques such as CoT [40] and s1 [27] has proven effective. Specifically for code, Z1 [43] explores dynamic adjustments to reasoning length to optimize generation quality. (ii) **Parallel Sampling and Voting:** Compared to single-pass generation, aggregating results from parallel sampling via Best-of-N selection [12] often yields significant performance gains. AlphaCode [19, 22] applies this strategy to code generation, leveraging large-scale parallel sampling and candidate selection to extend the capabilities of frontier models. (iii) **Iterative Refinement:** The model’s capacity for self-correction [25] aligns naturally with execution-guided debugging. In this paradigm, models leverage original or self-generated test cases and utilize execution feedback to iteratively refine and debug solutions [3, 5, 6]. Recent works have focused on hybridizing these dimensions to push the Pareto frontier of performance and cost. S* [20] introduces a unified framework that augments parallel sampling with serial debugging, while GenCluster [34] continuously scales test-time compute to IOI gold-level performance via parallel generation and behavioral clustering.

8 Threats to Validity

Limited benchmark coverage. Although we evaluate ESE and CAS on LiveCodeBench, which already provides a rigorous benchmark for recent competitive-programming tasks, this may limit the representativeness of our evaluation, future work can examine whether the same trends hold on broader real-world software engineering tasks.

Limited model and ensemble coverage. Although our experiments span three model families and include both within-family and cross-family two-model ensembles, this may limit the range of models and ensemble configurations, future work can study a wider

range of backbones, larger ensembles, and alternative ensemble constructions.

Limited semantic clustering method. Although we compare several clustering methods and find functional clustering induced by $\mathcal{T}_x^{gen}$ to be strongest in our setting, this may introduce bias due to the reliance on generated tests, future work can explore stronger generated tests or hybrid semantic validation strategies.

9 Conclusion

Existing uncertainty estimation methods often fail to distinguish between confident errors and correctness due to single-model overconfidence. To address this, we propose **Ensemble Semantic Entropy (ESE)**, which ensembles diverse models to estimate uncertainty and mitigate false positives. Experiments show that ESE correlates stronger with program correctness, improving prediction accuracy by **53.4%** in selective generation task. Furthermore, we propose an ESE-based cascading test-time scaling framework Cas, which reduces FLOPs by **64.9%** compared to single-model scaling while preserving performance.

References

- Aradhye Agarwal, Ayan Sengupta, and Tanmoy Chakraborty. 2025. The Art of Scaling Test-Time Compute for Large Language Models. arXiv:2512.02008 [cs.CL] <https://arxiv.org/abs/2512.02008>
- Anthropic. 2026. Claude Code: AI-powered coding assistant for developers. Product page. <https://claude.com/product/claude-code> Accessed: 2026-03-24.
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. arXiv:2207.10397 [cs.CL] <https://arxiv.org/abs/2207.10397>
- Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ktrw68Cmu9c>
- Jiefeng Chen, Jie Ren, Xinyun Chen, Chengrun Yang, Ruoxi Sun, Jinsung Yoon, and Seran Ö Arık. 2025. SETS: Leveraging Self-Verification and Self-Correction for Improved Test-Time Scaling. arXiv:2501.19306 [cs.AI] <https://arxiv.org/abs/2501.19306>
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128 [cs.CL] <https://arxiv.org/abs/2304.05128>
- Xiancai Chen, Zhengwei Tao, Kechi Zhang, Changzhi Zhou, Xinyu Zhang, Wanli Gu, Yuanpeng He, Mengdi Zhang, Xunliang Cai, Haiyan Zhao, and Zhi Jin. 2025. Revisit Self-Debugging with Self-Generated Tests for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 18003–18023. doi:10.18653/v1/2025.acl-long.881
- Cursor. 2026. Cursor Agent. Product page. <https://cursor.com/product> Accessed: 2026-03-24.
- Yihan Dai, Sijie Liang, Haotian Xu, Peichu Xie, and Sergey Mehtaev. 2025. Reducing Hallucinations in LLM-Generated Code via Semantic Triangulation. arXiv:2511.12288 [cs.SE] <https://arxiv.org/abs/2511.12288>
- Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C. Desmarais. 2023. Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing. arXiv:2308.16557 [cs.SE] <https://arxiv.org/abs/2308.16557>
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 8017 (June 2024), 625–630. doi:10.1038/s41586-024-07421-0
- Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Akshay Krishnamurthy, and Dylan J. Foster. 2025. Is Best-of-N the Best of Them? Coverage, Scaling, and Optimality in Inference-Time Alignment. arXiv:2503.21878 [cs.AI] <https://arxiv.org/abs/2503.21878>
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* 43, 2, Article 42 (Jan. 2025), 55 pages. doi:10.1145/3703155
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. arXiv:2403.07974 [cs.SE] <https://arxiv.org/abs/2403.07974>
- Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. 2025. Why Language Models Hallucinate. arXiv:2509.04664 [cs.CL] <https://arxiv.org/abs/2509.04664>
- Alex Kendall and Yarin Gal. 2017. What Uncertainties Do We Need in Bayesian Deep Learning for Computer Vision? arXiv:1703.04977 [cs.CV] <https://arxiv.org/abs/1703.04977>
- Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. 2023. Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. arXiv:2302.09664 [cs.CL] <https://arxiv.org/abs/2302.09664>
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. 2017. Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. arXiv:1612.01474 [stat.ML] <https://arxiv.org/abs/1612.01474>
- Rémi Leblond, Felix Gimeno, and Florent Althé. 2023. AlphaCode 2 Technical Report. https://storage.googleapis.com/deepmind-media/AlphaCode2/AlphaCode2_Tech_Report.pdf
- Dacheng Li, Shiyi Cao, Chengkun Cao, Xiuyu Li, Shangyin Tan, Kurt Keutzer, Jiarong Xing, Joseph E. Gonzalez, and Ion Stoica. 2025. S*: Test Time Scaling for Code Generation. arXiv:2502.14382 [cs.LG] <https://arxiv.org/abs/2502.14382>
- Kefan Li and Yuan Yuan. 2024. Large Language Models as Test Case Generators: Performance Evaluation and Enhancement. arXiv:2404.13340 [cs.SE] <https://arxiv.org/abs/2404.13340>
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustín Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158
- Yunhao Liang, Ruixuan Ying, Takuya Taniguchi, Chengguang Gan, and Zhe Cui. 2025. RECODE: Leveraging Reliable Self-generated Tests and Fine-Grained Execution Feedback to Enhance LLM-Based Code Generation. In *Advanced Intelligent Computing Technology and Applications*, De-Shuang Huang, Bo Li, Haiming Chen, and Chuanlei Zhang (Eds.). Springer Nature Singapore, Singapore, 510–521.
- Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. arXiv:2305.01210 [cs.SE] <https://arxiv.org/abs/2305.01210>
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 [cs.CL] <https://arxiv.org/abs/2303.17651>
- Andrey Malinin and Mark Gales. 2021. Uncertainty Estimation in Autoregressive Structured Prediction. arXiv:2002.07650 [stat.ML] <https://arxiv.org/abs/2002.07650>
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. arXiv:2501.19393 [cs.CL] <https://arxiv.org/abs/2501.19393>
- Dang Nguyen, Ali Payani, and Baharan Mirzasoleiman. 2025. Beyond Semantic Entropy: Boosting LLM Uncertainty Quantification with Pairwise Semantic Similarity. arXiv:2506.00245 [cs.LG] <https://arxiv.org/abs/2506.00245>
- OpenAI, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, Che Chang, Kai Chen, Mark Chen, Enoch Cheung, Aidan Clark, Dan Cook, Marat Dukhan, Casey Dvorak, Kevin Fives, Vlad Fomenko, Timur Garipov, Kristian Georgiev, Mia Glaese, Tarun Gogineni, Adam Goucher, Lukas Gross, Katia Gil Guzman, John Hallman, Jackie Hehir, Johannes Heidecke, Alec Helyar, Haitang Hu, Romain Huet, Jacob Huh, Saachi Jain, Zach Johnson, Chris Koch, Irina Kofman, Dominik Kundel, Jason Kwon, Volodymyr Kyrilov, Elaine Ya Le, Guillaume Leclerc, James Park Lennon, Scott Lessans, Mario Lezcano-Casado, Yuanzhi Li, Zhuohan Li, Ji Lin, Jordan Liss, Lily Liu, Jiancheng Liu, Kevin Lu, Chris Lu, Zoran Martinovic, Lindsay McCallum, Josh McGrath, Scott McKinney, Aidan McLaughlin, Song Mei, Steve Mostovoy, Tong Mu, Gideon Myles, Alexander Neitz, Alex Nichol, Jakub Pachocki, Alex Paino, Dana Palmie, Ashley Pantuliano, Giambattista Parascandolo, Jongsoo Park, Leher Pathak, Carolina Paz, Ludovic Peran, Dmitry Pimenov, Michelle Pokrass, Elizabeth Proehl, Huida Qiu, Gaby Raila, Filippo Raso, Hongyu Ren, Kimmy Richardson, David Robinson, Bob Rotsted, Hadi Salman, Suvansh Sanjeev, Max Schwarzer, D. Sculley, Harshit Sikchi, Kendal Simon, Karan Singhal, Yang Song, Dane Stuckey, Zhiqing Sun, Philippe Tillet, Sam Toizer, Foivos Tsimpourlas, Nikhil Vyas, Eric Wallace, Xin

- Wang, Miles Wang, Olivia Watkins, Kevin Weil, Amy Wendling, Kevin Whinnery, Cedric Whitney, Hannah Wong, Lin Yang, Yu Yang, Michihiro Yasunaga, Kristen Ying, Wojciech Zaremba, Wenting Zhan, Cyril Zhang, Brian Zhang, Eddie Zhang, and Shengjia Zhao. 2025. gpt-oss-120b & gpt-oss-20b Model Card. arXiv:2508.10925 [cs.CL] <https://arxiv.org/abs/2508.10925>
- [30] Ji Won Park and Kyunghyun Cho. 2026. Efficient semantic uncertainty quantification in language models via diversity-steered sampling. arXiv:2510.21310 [cs.CL] <https://arxiv.org/abs/2510.21310>
- [31] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv. <https://www.microsoft.com/en-us/research/publication/the-impact-of-ai-on-developer-productivity-evidence-from-github-copilot/>
- [32] Chaitanya Ravuri and Saman Amarasinghe. 2025. Eliminating Hallucination-Induced Errors in LLM Code Generation with Functional Clustering. arXiv:2506.11021 [cs.SE] <https://arxiv.org/abs/2506.11021>
- [33] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: a Method for Automatic Evaluation of Code Synthesis. arXiv:2009.10297 [cs.SE] <https://arxiv.org/abs/2009.10297>
- [34] Mehrzad Samadi, Aleksander Ficek, Sean Narenthiran, Siddhartha Jain, Wasi Uddin Ahmad, Somshubra Majumdar, Vahid Noroozi, and Boris Ginsburg. 2025. Scaling Test-Time Compute to Achieve IOI Gold Medal with Open-Weight Models. arXiv:2510.14232 [cs.LG] <https://arxiv.org/abs/2510.14232>
- [35] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2023. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. arXiv:2302.06527 [cs.SE] <https://arxiv.org/abs/2302.06527>
- [36] Arindam Sharma and Cristina David. 2025. Assessing Correctness in LLM-Based Code Generation via Uncertainty Estimation. arXiv:2502.11620 [cs.SE] <https://arxiv.org/abs/2502.11620>
- [37] Adi Simhi, Itay Itzhak, Fazl Barez, Gabriel Stanovsky, and Yonatan Belinkov. 2025. Trust Me, I'm Wrong: LLMs Hallucinate with Certainty Despite Knowing the Answer. arXiv:2502.12964 [cs.CL] <https://arxiv.org/abs/2502.12964>
- [38] Team GLM, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, Hao Yu, Hongning Wang, Jiadao Sun, Jiajie Zhang, Jiale Cheng, Jiayi Gui, Jie Tang, Jing Zhang, Jingyu Sun, Juanzi Li, Lei Zhao, Lindong Wu, Lucen Zhong, Mingdao Liu, Minlie Huang, Peng Zhang, Qinkai Zheng, Rui Lu, Shuaiqi Duan, Shudan Zhang, Shulin Cao, Shuxun Yang, Weng Lam Tam, Wenyi Zhao, Xiao Liu, Xiao Xia, Xiaohan Zhang, Xiaotao Gu, Xin Lv, Xinghan Liu, Xinyi Liu, Xinyue Yang, Xixuan Song, Xunkai Zhang, Yifan An, Yifan Xu, Yilin Niu, Yuantao Yang, Yueyan Li, Yushi Bai, Yuxiao Dong, Zehan Qi, Zhaoyu Wang, Zhen Yang, Zhengxiao Du, Zhenyu Hou, and Zihan Wang. 2024. ChatGLM: A Family of Large Language Models from GLM-130B to GLM-4 All Tools. arXiv:2406.12793 [cs.CL] <https://arxiv.org/abs/2406.12793>
- [39] Thomas Valentin, Ardi Madadi, Gaetano Sapia, and Marcel Böhme. 2025. Incoherence as Oracle-less Measure of Error in LLM-Based Code Generation. arXiv:2507.00057 [cs.PL] <https://arxiv.org/abs/2507.00057>
- [40] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2023. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL] <https://arxiv.org/abs/2201.11903>
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL] <https://arxiv.org/abs/2505.09388>
- [42] Zhuoyi Yang, Xu Guo, Tong Zhang, Huijuan Xu, and Boyang Li. 2025. Test-time Scaling of LLMs: A Survey from A Subproblem Structure Perspective. arXiv:2511.14772 [cs.CL] <https://arxiv.org/abs/2511.14772>
- [43] Zhaojian Yu, Yinghao Wu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. 2025. Z1: Efficient Test-time Scaling with Code. arXiv:2504.00810 [cs.CL] <https://arxiv.org/abs/2504.00810>
- [44] Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. 2024. Large Language Model Cascades with Mixture of Thoughts Representations for Cost-efficient Reasoning. arXiv:2310.03094 [cs.CL] <https://arxiv.org/abs/2310.03094>
- [45] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Shi, Yuchi Ma, Wanjuan Zhong, Jiachi Chen, Mingzhi Mao, and Zibin Zheng. 2025. LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA022 (June 2025), 23 pages. doi:10.1145/3728894